Babel-formal: Translation of Proofs Between Lean and Rocq

Anonymous Authors[1]

Abstract

In this work, we investigate using proof terms (the low-level representation of formal proofs) as a pivot language for translating proof scripts between proof assistants and across tactic sets. Unlike direct proof translation, this approach does not require an aligned training corpus; it only needs aligned context at inference time so that both systems elaborate comparable terms. We compare two strategies: (1) direct script-to-script translation with two off-the-shelf LLMs (GPT-5, Qwen2.5-Coder-32B-Instruct), and (2) proof term translation, where an LLM turns a proof term into a proof script in the target language. We build a small benchmark of aligned sources (14 files, 117 lemmas) across Lean and Rocq, and train models to map Lean and Rocq proof terms back to their native proof scripts. Our experiments show that proof term translation works for cross-assistant translation and for translation between tactic sets. It is complementary to using an off-the-shelf LLM for direct proof script translation, scales easily in terms of training data, and handles tactic-set translation better (*e.g.,* vanilla Rocq → SSReflect).

1. Introduction

Proof assistants such as Rocq and Lean accumulate over time large libraries of formal mathematics (for example, MathComp (Mahboubi & Tassi, 2020) for Rocq and Mathlib (Community, 2019) for Lean), yet proofs are not reusable across systems, even in a strictly aligned context. We study two translation settings. The first is *cross-assistant* translation between Lean and Rocq, which is useful to import the many results that exist in one prover but not in the other. The second is *cross-tactic* translation inside Rocq, from the standard Rocq tactic language, called here *vanilla Rocq*, to *SSReflect* (Gonthier et al.), which is a compact, structured tactic language widely used with MathComp (see an example in Appendix A.5), and can make existing proofs more robust to changes.

These translation capabilities matter in practice because large verified libraries are not portable: many results exist in one ecosystem but not the other, and re-proving them is costly even when statements are aligned. Beyond reuse, translation also supports migration of existing developments toward more robust proof styles (*e.g.,* vanilla Rocq → SSReflect) without rewriting whole files. Finally, our approach is complementary to statement translation. Proof term translation can then fill this gap: once statements are aligned, we can leverage the source proof term to synthesize a proof script that the target assistant can check.

Both Rocq and Lean provide a way to pretty print *proof terms* given by the proof assistant (see Figure 1) in a way that omits details while keeping the structure of a proof. We thus explore using *pretty-printed proof terms* as a pivot language. This removes the need for an aligned training corpus of tactic scripts. At inference time, we only require strictly aligned statements, *i.e.,* the same theorem and premises, so that both systems elaborate comparable terms. The approach is related to decompilation in programming languages, where models map assembly to higher-level C programs (Tan et al., 2024).

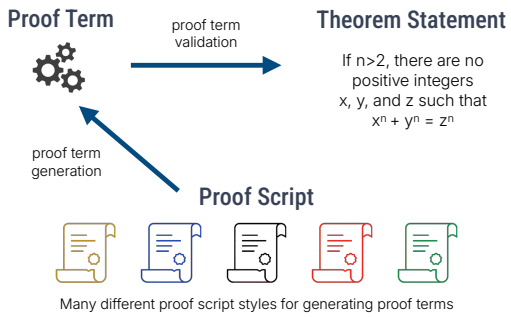
We compare two strategies:

- **Direct script translation.** An LLM is prompted to convert a proof script directly from source tactics to target tactics. We apply this in both settings (Lean ↔ Rocq and vanilla Rocq → SSReflect), using GPT-5 (OpenAI, 2025) as a baseline.
- **Proof term translation.** We first extract the proof term, then train models to predict scripts from proof terms (Lean term to Lean script, Rocq term to Rocq script). At inference, we extract the term for the source proof and use the model to produce a script in the target assistant or tactic set (see Figure 4). For an example of Lean → Rocq inference see Appendix A.6, and Appendix A.5 for vanilla Rocq → SSReflect.

To support this comparison, we build a benchmark of 14 files

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.



Given a formal statement of a theorem, proving it amounts to constructing a proof term, *i.e.,* a precise object that the prover can type-check for correctness. However, these proof terms are typically extremely verbose, capturing every low-level detail, and are impractical to write directly by hand.
To address this, users interact with interactive theorem provers through proof scripts, meta-programs designed to generate proof terms. Over time, this layer has evolved, leading to a diverse ecosystem of proof script languages (tactic languages).

Figure 1. How interactive theorem provers operate.

with 117 lemmas that are strictly aligned between Lean and Rocq. We also mention, but do not pursue, direct low-level term translation between Lean and Rocq (rocq-lean-import Development Team., 2020). The main drawback of that direction is that the output is not tied to a maintainable proof script.

Our main contributions are:

- **Two term-to-script models**, trained to reconstruct proof scripts from pretty-printed proof terms. *Babel-translate* is a single term-to-script model trained on both (Lean term→Lean script) and (Rocq term→Rocq script); at inference, we perform cross-assistant translation by prompting it to emit tactics in the target language while providing the source term. *Babel-ssreflect* translates Rocq proof terms directly into SSReflect scripts, enabling cross-tactic translation from vanilla Rocq proofs to SSReflect within Rocq.
- **A small aligned benchmark** (14 files, 117 lemmas) between Lean and Rocq, with an evaluation of cross-assistant and cross-tactic translation.

2. Related Work

TODO

3. Methodology

We describe how we build the training data, the inference pipeline, and benchmarks. To build the training data we follow four steps (Muennighoff et al., 2025): (1) select a diverse subset of proofs, (2) extract proof terms in Rocq and Lean, (3) generate synthetic backward reasoning traces inferring proof script from proof term, and (4) train models that predict both tactic script and reasoning trace from a proof term(Figure 2). Full prompt templates and training details are in Appendices A.1 and A.2.

Proof Subset Selection. Following S1 (Muennighoff et al., 2025), we create in each case a training set of 1000 entries. For translation between Lean and Rocq, we select a subset of 500 diverse proofs in C-CoRN (Cruz-Filipe et al., 2004) and 500 diverse proofs in Mathlib. For translation from vanilla Rocq to SSReflect, we select a subset of 1000 entries in MathComp. In both cases, we filter by length, then choose a diverse subset (statements and proofs) using `BM25` (Robertson & Jones, 1976).

Proof Term Extraction. For Rocq, we use `coqpyt` (Carrott et al., 2024) to extract, for each proof, its proof term together with the premises and notation declarations. For Lean 4, we use `leanclient` (Dressler, 2025) to extract the proof term and the context.

Backward Reasoning Trace Generation. Given each proof term and the corresponding target script, we generate backward reasoning traces with Gemini 2.5 Pro (DeepMind, 2024). Backward reasoning trace generation refers here to prompting the LLM with the proof term and the target proof script, and asking it to generate a reasoning process that works backwards from the script. The resulting training dataset follows the structure described in Figure 2.

Benchmarks. We evaluate on two benchmarks:

Cross-assistant (Lean ↔ Rocq). We created 14 files with the same statements in Rocq and Lean, for a total of 117 aligned lemmas. Preparing files took about 20 hours of manual effort in total. Each lemma is proved natively in both systems, so the proof scripts differ but are type-correct in aligned contexts. We call two statements *aligned* if they express the same mathematical content under a fixed mapping of definitions and notations, and if they type-check in aligned local contexts (*i.e.,* same premises and imports up to that mapping). In practice, we created paired Rocq/Lean files by porting statements lemma-by-lemma, keeping the dependency surface small and explicit, and checking that both files compile. From these, we extracted comparable proof terms in both systems. An example of such a source file is in Appendix A.4.

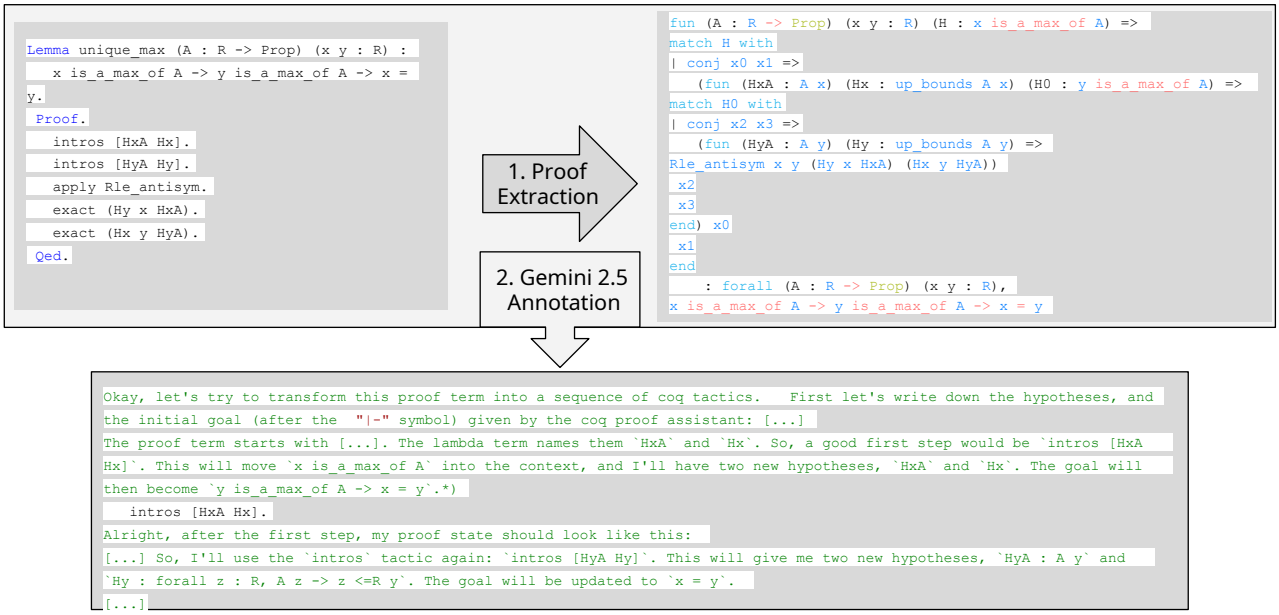


Figure 2. Last two steps of dataset collection.

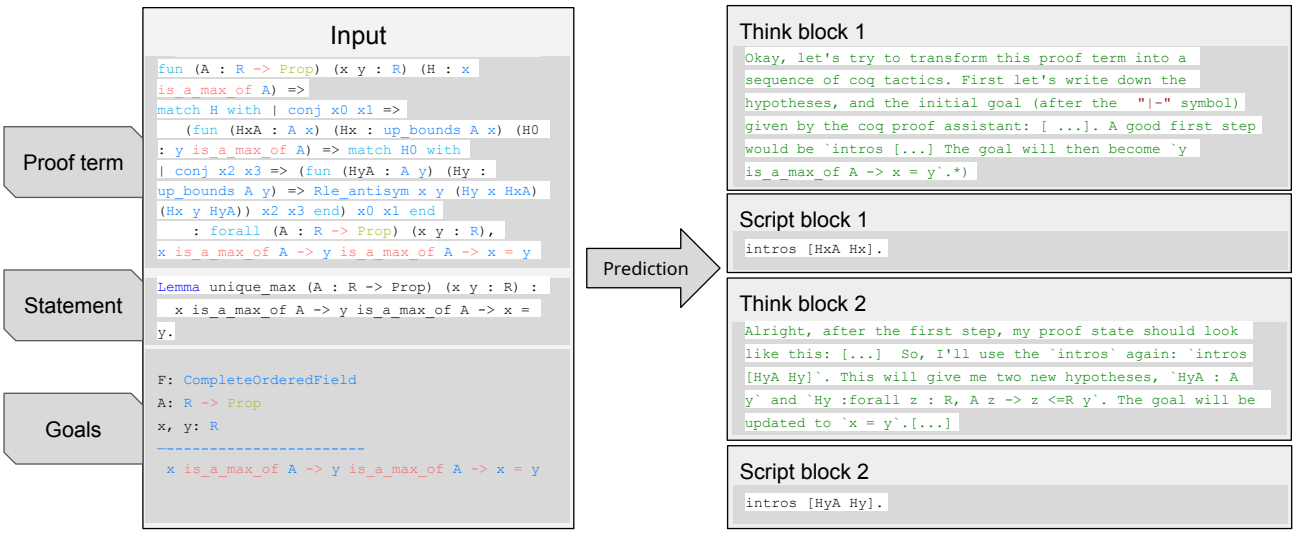


Figure 3. Training.

Cross-tactic (vanilla Rocq → SSReflect). We collect 500 Rocq proofs from the C-CoRN library (Cruz-Filipe et al., 2004), which is developed in vanilla Rocq (in contrast to MathComp, which is built on SSReflect). We use these proofs to assess translation from vanilla Rocq tactics to SSReflect. To ensure diversity, we pick subsets maximizing document distance (via BM25). An example entry is in Appendix A.5.

Limitations. Our aligned benchmark is small and manually constructed. We view it as a controlled testbed for translation mechanisms rather than a claim of broad coverage.

Inference Pipeline. At inference, we leverage the source proof term and use the appropriate strategy to obtain a script in the target assistant or tactic set (Figure 4). In the *cross-tactic* regime (vanilla Rocq → SSReflect), inference matches training: *Babel-ssreflect* is trained to map Rocq proof terms directly to SSReflect scripts in the same Rocq context.

In the *cross-assistant* regime (Lean ↔ Rocq), training is *native* (Lean term → Lean script; Rocq term → Rocq script),

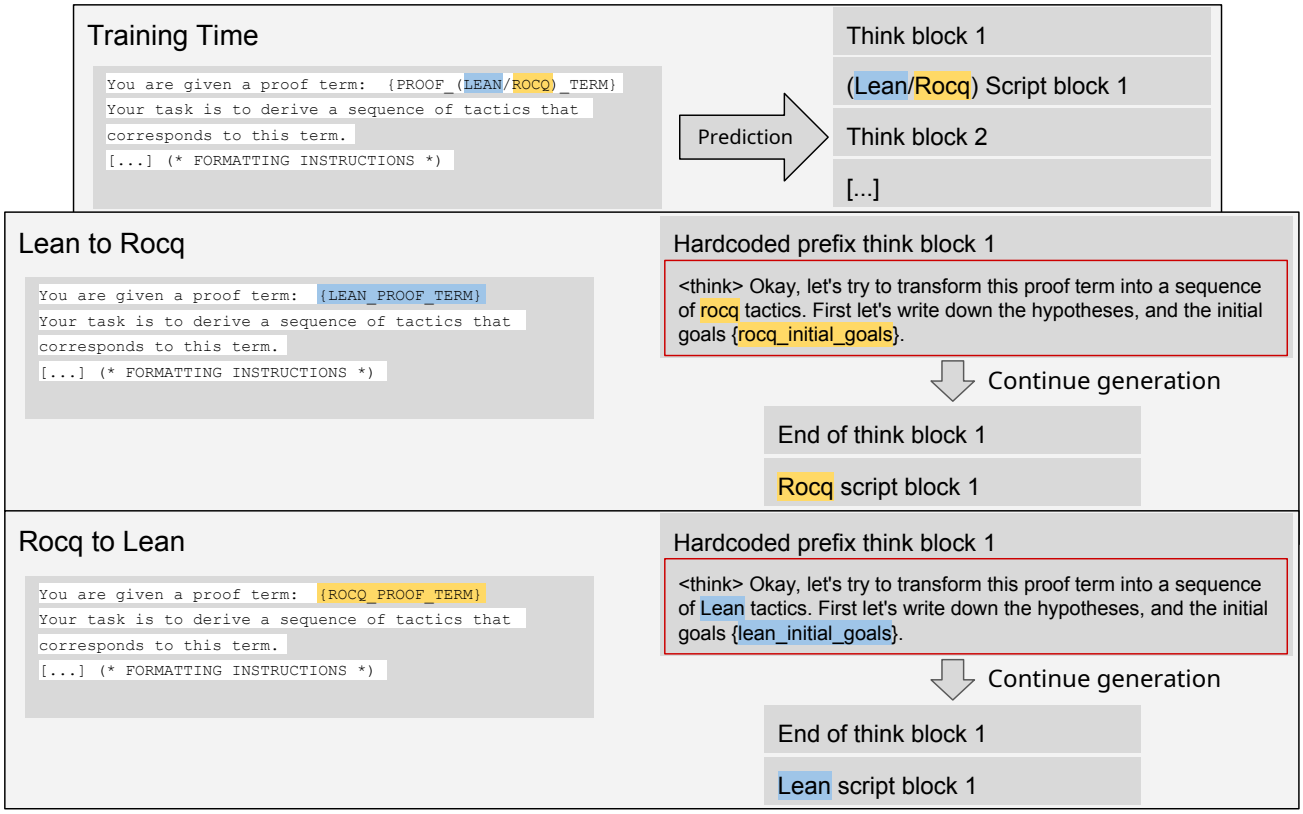


Figure 4. Inference strategy.

but inference is *cross*. We enable this by supplying the model with the *target* statement and an *aligned* target context (premises, notations), so that the source term can be interpreted as a proof of the target goal. In practice, we pretty-print the source term, instantiate the prompt with the target prover's goal/context, and ask the model to produce a target script that reconstructs the same term structure under this alignment.

Models and Training. We fine-tune `Qwen2.5-Coder-32B-Instruct` (Hui et al., 2024) to translate a proof term into a tactic script. We train two models: (i) **Babel-translate:** Lean term → Lean script and Rocq term → Rocq script; and (ii) **Babel-ssreflect:** Rocq term → SSReflect script. These models are used at inference to produce target scripts from source terms (Figure 4).

Evaluation. We evaluate the three translation directions defined in Section 3. In the *cross-assistant* setting (Lean ↔ Rocq), the model is given a source proof (script for direct translation, or a pretty-printed proof term for Babel) together with the *aligned* target statement and local context; success means the generated target script type-checks in the *target* prover. In the *cross-tactic* setting (vanilla Rocq → SSReflect), the prover and statements are unchanged and only the tactic language changes; success means the SSReflect script type-checks in Rocq. We verify scripts using Rocq tooling (Pétanque) (Team., 2024; Teodorescu et al., 2024) and Lean tooling (leanclient) (Dressler, 2025).

We report verification success under two decoding regimes (see Figure 4).

Sampling. We draw k independent candidate scripts and report pass@k (a problem is solved if and only if at least one candidate verifies). Decoding can optionally query the prover during generation and append goals and/or error messages (see the feedback modes in Section 4); the main table reports pass@k under a fixed sampling budget with a bounded amount of such feedback.

Interactive repair. For direct script→script baselines (GPT-5 and Qwen2.5-Coder-32B-Instruct), we run an error-driven agent for up to 4 repair rounds.

Tasks. We evaluate three directions: (i) Lean → Rocq, (ii) Rocq → Lean (cross-assistant), and (iii) vanilla Rocq → SSReflect (cross-tactic), using the benchmarks in Section 3. For (i) and (ii) we apply *Babel-translate* (proof term → native script) and compare with direct GPT-5 and base model (Qwen2.5-32B-coder-Instruct) translation. For (iii) we ap-

Table 1. Benchmark results (%) across three tasks.

Method	Lean to Rocq	Rocq to Lean	Rocq to SSReflect
Term → Script translation			
Babel-translate (pass@256 x 2 feedback)	**88.0**	**59.8**	–
Babel-ssreflect (pass@256 x 2 feedback)	–	–	**33.2**
Qwen2.5-Coder-32B (pass@256 x 2 feedback)	52.1	32.5	0.0
Direct Script → Script translation			
GPT-5 (@1 x 4 feedback)	**82.9**	**67.5**	**16.6**
Qwen2.5-Coder-32B (@128 x 4 feedback)	25.6	39.3	1.7
Ensemble translation			
GPT-5 (@1 x 4 feedback) + Babel(pass@256 x 2 feedback)	**93.2**	**83.7**	**34.0**

ply *Babel-ssreflect* (proof term → SSReflect script), again comparing with direct GPT-5 and base model translation.

4. Results and Analysis

For direct proof script translation, GPT-5 reaches 82.9% success for Lean → Rocq and 67.5% for Rocq → Lean, while it achieves 16.6% on the cross-tactic task vanilla Rocq → SSReflect. In contrast, the term-based cross-tactic model attains 33.3% on Rocq → SSReflect, showing a clear gap for tactic-style transfer. For cross-assistant translation, term-to-script models remain competitive (88.0% Lean → Rocq; 59.8% Rocq → Lean).

Ablation study. Removing the reasoning component in *Babel-ssreflect* reduces Rocq → SSReflect from 33.2% to 21.4%. The model is trained to output only the tactic script. At inference time, we use the same pipeline, but we prompt for tactic-only outputs. For Lean ↔ Rocq, the model trained without reasoning produced mixed scripts that were not usable, so we do not report scores.

Discrepancy between Lean and Rocq results. We observe an asymmetry between Lean → Rocq and Rocq → Lean translation results. A plausible explanation is that the two proof assistants expose different feedback signals to an interactive agent. Rocq typically checks scripts sentence-by-sentence and reports errors at the first failing command together with a closely related proof state, yielding localized repair signals.

Inference strategy. We evaluate inference-time scaling with different levels of tool feedback. Although Babel is not trained with interactive feedback, we can query the prover during generation and append the resulting goals and/or error messages to the LLM context before sampling the next continuation. We consider the following modes:

- **Whole proof generation.** Generate an entire proof without intermediate feedback; report success/failure at the end.
- **Goals.** After each tactic, query the prover and append the resulting goals before generating the next step.
- **Errors.** On failure, rollback to the end of the last `think` block, append the error message and the attempted tactic(s), and continue.
- **Goals & errors.** Combine **Goals** and **Errors**.
- **No feedback.** Do not return goals or errors; on failure, delete the last `think` block and tactic, and resample.

Scaling laws. We measure how translation success scales with inference-time compute. For each theorem, we sample up to k candidate scripts and report *pass*@k (solved if any sample verifies). Figure 5 compares feedback modes, and Figure 6 compares Babel to a direct script-translation baseline under a matched sampling budget.

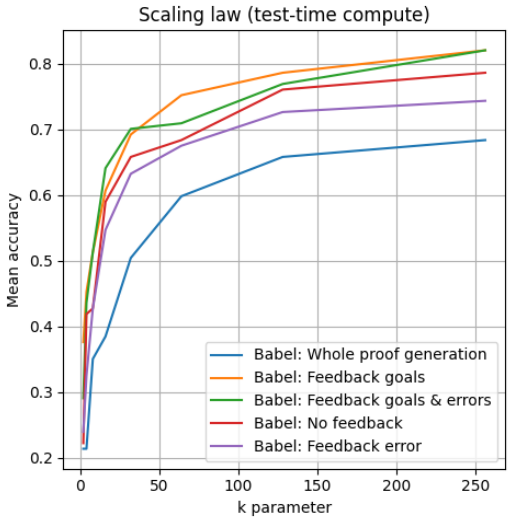


Figure 5. Lean → Rocq scaling (pass@k) for different feedback modes.

5. Conclusion and Future Work

We introduced proof term translation as a way to translate proofs across assistants and tactic styles. Unlike direct

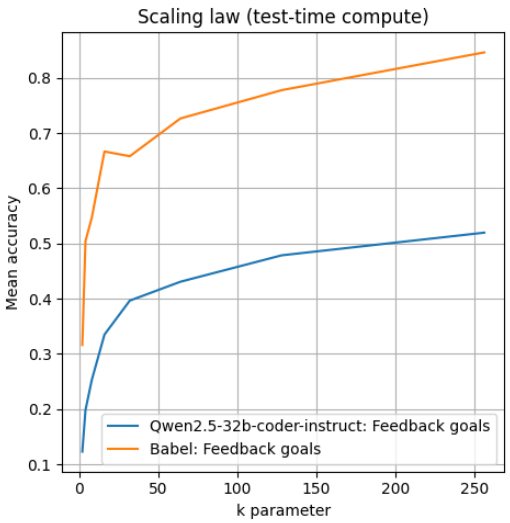


Figure 6. Lean → Rocq scaling (pass@k): Babel vs. Qwen2.5-Coder-32B-Instruct, with matched sampling budget in the Term → Script mode.

tactic translation, this approach does not need parallel scripts for training. On our benchmarks it brings clear gains for style transfer and remains competitive for cross-assistant translation.

Future work. We see three directions:

- **Scale.** Train on larger corpora and more domains, since the approach does not require aligned scripts.
- **Multilingual.** Scaling this approach to additional assistants mainly requires: (i) extraction and pretty-printing of proof terms, (ii) an automated verification in the target system. Training data is then easy to obtain from native corpora because we do not require parallel scripts.
- **Source-to-source translation (Lean ↔ Rocq).** Use off-the-shelf LLMs for source-to-source translation, and bootstrap proof script translation to check whether the source translation is correct.

Impact Statement

This paper studies automated translation of formal proofs between proof assistants and between tactic languages. A primary positive impact is improved interoperability between formal-mathematics ecosystems, which can reduce duplicated effort and accelerate the reuse of verified results. Because all translated scripts are ultimately checked by the target proof assistant, the approach is designed to mitigate the risk of producing incorrect proofs that appear superficially plausible. Remaining risks include engineering misuse (e.g., over-trusting generated scripts before verification, or deploying translation pipelines without adequate checks) and potential shifts in how contributors learn and write proofs. We therefore emphasize verification-first workflows and recommend that tool support surface proof obligations, errors, and provenance of generated artifacts.

References

Carrott, P., Saavedra, N., Thompson, K., Lerner, S., Ferreira, J. F., and First, E. CoqPyt: Proof navigation in python in the era of LLMs. *arXiv preprint arXiv:2405.04282*, 2024. URL https://arxiv.org/abs/2405.04282.

Community, M. The lean mathematical library. *arXiv preprint arXiv:1912.09375*, 2019. URL https://arxiv.org/abs/1912.09375.

Cruz-Filipe, L., Geuvers, H., and Wiedijk, F. C-corn, the constructive coq repository at nijmegen. In *Mathematical Knowledge Management (MKM 2004)*, volume 3119 of *Lecture Notes in Computer Science*, pp. 88–103. Springer, 2004. doi: 10.1007/978-3-540-27818-4_7.

DeepMind, G. Gemini 2.5 Pro: Advanced multimodal model. https://ai.googleblog.com/2024/07/gemini-25-our-newest-gemini-model-with.html, 2024.

Dressler, O. Leanclient: Python client to interact with the lean4 language server, 1 2025. URL https://github.com/oOo0oOo/leanclient.

Gonthier, G., Mahboubi, A., and Tassi, E. Ssreflect: Small scale reflection in Coq. Coq documentation. URL https://coq.inria.fr/refman/addendum/ssreflect.html.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Lu, K., Dang, K., Fan, Y., Zhang, Y., Yang, A., Men, R., Huang, F., Zheng, B., Miao, Y., Quan, S., Feng, Y., Ren, X., Ren, X., Zhou, J., and Lin, J. Qwen2.5-Coder technical report. 2024.

Mahboubi, A. and Tassi, E. Mathematical components. https://math-comp.github.io, 2020.

Muennighoff, N., Yang, Z., Shi, W., Li, X. L., Li, F.-F., Hajishirzi, H., Zettlemoyer, L., Liang, P., Candès, E., and Hashimoto, T. S1: Simple test-time scaling. *arXiv preprint arXiv:2405.00002*, 2025.

OpenAI. Introducing GPT-5. https://openai.com/blog/introducing-gpt-5, 2025.

Robertson, S. E. and Jones, K. S. Relevance weighting of search terms. *Journal of the American Society for Information Science*, 27(3):129–146, May 1976. ISSN

1097-4571. doi: 10.1002/asi.4630270302. URL http://dx.doi.org/10.1002/asi.4630270302.

rocq-lean-import Development Team., T. Coq is a lean typechecker. https://github.com/rocq-community/rocq-lean-import, 2020.

Tan, H., Luo, Q., Li, J., and Zhang, Y. Llm4decompile: Decompiling binary code with large language models. *arXiv preprint arXiv:2406.00004*, 2024.

Team., T. C. L. D. Coq-LSP: Language server protocol implementation for the Coq proof assistant. https://github.com/ejgallego/coq-lsp, 2024.

Teodorescu, L., Baudart, G., Arias, E. J. G., and Lelarge, M. Nlir: Natural language intermediate representation for mechanized theorem proving. In *MathAI@NeurIPS*, 2024.

A. Appendix

A.1. Training Details

We fine-tune a pretrained and instruction-tuned `Qwen2.5-32b-Coder-Instruct`(Hui et al., 2024) for reasoning, following the setup of S1 simple test-time scaling (Muennighoff et al., 2025). We train for five epochs with a global batch size of 16, which yields 315 gradient steps. We use bfloat16 precision and an AdamW optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.95$, and weight decay 1×10^{-4}. The learning rate is 1×10^{-5}, warmed up linearly for the first 5% (16 steps) and then cosine-decayed to zero over the remaining 299 steps. We compute loss only on the reasoning traces and the final solutions, and we choose a sequence length large enough to avoid truncation. On GENCI hardware (8×4 NVIDIA H100), this run completes in about one hour. At inference, we sample up to 256 candidates.

A.2. Prompt Babel-translate

For Lean → Rocq, we instruct the model to translate a Lean proof term and its local context into a Rocq tactic script. We prepend the following instruction:

```
You are given a proof term:

{term}

Your task is to derive a sequence of tactics that corresponds to this term.

When you work through the problem, write down your reasoning in detail inside
<think> ... </think> tags. This reasoning should reflect your natural thought
process as you explore the structure of the term and figure out what tactics
to apply. You should consider different possible approaches, reflect on why
some might or might not work, and gradually converge on a tactic choice.

After each reasoning block, provide the next (group of) tactic(s) enclosed in:

\box{{
  <tactic>
}}

Some dependencies that could be helpful:

{dependencies}
```

We hard-code the beginning of the first thinking block:

```
<think> Okay, let's try to transform this proof term into a sequence of coq
tactics.
First let's write down the hypotheses, and the initial goal
(after the "|-" symbol) given by the coq proof assistant:
{initial_goal}.
```

A.3. Prompt GPT-5 direct translation

For Lean → Rocq, we instruct the model to translate a Lean proof script into a Rocq proof script. We give the following instruction:

```
You are a precise translator from Lean 4 proofs to Rocq (Coq).

## INPUT
```

```
<ROCQ_STATEMENT>
{rocq_statement}
</ROCQ_STATEMENT>

<LEAN_STATEMENT>
{lean_statement}
</LEAN_STATEMENT>

<LEAN_PROOF>
{lean_proof}
</LEAN_PROOF>

<DEPENDENCIES>
{dependencies}
</DEPENDENCIES>

## RULES
1) Produce two blocks in this exact order and format:
a) A thought process.
b) The final Rocq proof script delimited by the markers below.
2) Use only lemmas/results listed in <DEPENDENCIES>. Do not rely on
external lemmas beyond these. Standard Rocq/Coq tactics are allowed.
3) Do not invent new lemmas.
4) Do not use "Admitted.". End with "Qed.".
5) Output must contain only the two blocks below, nothing else.

## OUTPUT FORMAT (STRICT)
<BEGIN_RATIONALE>
[Thought process.]
</END_RATIONALE>

<BEGIN_ROCQ_PROOF>
[Place only the Rocq/Coq script here. Inside "Proof." ... "Qed.",
do not rewrite the Rocq statement.]
</END_ROCQ_PROOF>

Now produce the output for the given input.
```

In case of an issue, we prepend the following feedback to the next prompt:

```
You already tried this proof:
{proof}
but it fails with this error:
{error_message}.
```

A.4. Benchmark example

We show one Lean/Rocq pair from the aligned set. The Rocq entry is:

```
Class AbsField := {
 R0   : Type;
 zero : R0;
 one  : R0;
```

```
  add  : R0 -> R0 -> R0;
  mul  : R0 -> R0 -> R0;
  opp  : R0 -> R0;
  abs  : R0 -> R0;
  leR  : R0 -> R0 -> Prop;
  ltR  : R0 -> R0 -> Prop;


  N0       : Type;
  Nle      : N0 -> N0 -> Prop;
  Nmax     : N0 -> N0 -> N0;
  Nle_max_l : forall x y, Nle x (Nmax x y);
  Nle_max_r : forall x y, Nle y (Nmax x y);

  add_comm  : forall x y, add x y = add y x;
  add_assoc : forall x y z, add (add x y) z = add x (add y z);
  add_zero  : forall x, add x zero = x;
  add_opp   : forall x, add x (opp x) = zero;
  mul_comm  : forall x y, mul x y = mul y x;
  mul_assoc : forall x y z, mul (mul x y) z = mul x (mul y z);
  mul_one   : forall x, mul x one = x;

  le_refl   : forall x, leR x x;
  le_trans  : forall x y z, leR x y -> leR y z -> leR x z;
  add_le_add : forall a b c d, leR a b -> leR c d -> leR (add a c)\\
  (add b d);

  abs_nonneg : forall x, leR zero (abs x);
  abs_triangle : forall x y, leR (abs (add x y)) (add (abs x) (abs y));
  abs_opp : forall x, abs (opp x) = abs x;
  abs_sub_symm : forall x y, abs (add x (opp y)) = abs (add y (opp x));
  sub_decomp : forall x y z, add x (opp z) = add (add x (opp y)) \\
  (add y (opp z));
  sub_eq_zero : forall x y, add x (opp y) = zero -> x = y;


  eq_of_forall_eps2 : forall x, (forall eps, ltR zero eps -> leR \\
  (abs x) (add eps eps)) -> x = zero
}.

Section Limits.
Context {A : AbsField}.

Definition sub (x y : R0) : R0 := add x (opp y).

Definition limit (u : N0 -> R0) (l : R0) : Prop :=
forall eps, ltR zero eps -> exists N, forall n, Nle N n -> leR\\
(abs (sub (u n) l)) eps.

Lemma abs_sub_triangle (x y z : R0) :
leR (abs (sub x z)) (add (abs (sub x y)) (abs (sub y z))).
Proof.
unfold sub.
rewrite (sub_decomp x y z).
```

```
apply abs_triangle.
Qed.

Theorem limit_unique (u : N0 -> R0) (l m : R0) :
limit u l -> limit u m -> l = m.
Proof.
intros Hl Hm.
assert (Hbound: forall eps, ltR zero eps -> leR (abs (sub l m))\\
(add eps eps)).
{ intros eps Heps.
 destruct (Hl eps Heps) as [N1 HN1].
 destruct (Hm eps Heps) as [N2 HN2].
 set (N := Nmax N1 N2).
 assert (H1u: leR (abs (sub (u N) l)) eps).
 { apply HN1. apply Nle_max_l. }
 assert (H1: leR (abs (sub l (u N))) eps).
 { unfold sub in H1u. rewrite abs_sub_symm in H1u. fold sub\\
  in H1u. exact H1u. }
 assert (H2: leR (abs (sub (u N) m)) eps).
 { apply HN2. apply Nle_max_r. }
 assert (Htri: leR (abs (sub l m)) (add (abs (sub l (u N)))\\
 (abs (sub (u N) m)))) by apply abs_sub_triangle.
 eapply le_trans; [exact Htri|].
 apply add_le_add; [exact H1|exact H2].
}
assert (Hz: sub l m = zero).
{ apply eq_of_forall_eps2. exact Hbound. }
unfold sub in Hz.
now apply sub_eq_zero in Hz.
Qed.

End Limits.
```

The Lean counterpart is:

```
class AbsField (R : Type) where
zero  : R
one   : R
add   : R → R → R
mul   : R → R → R
opp   : R → R
abs   : R → R
leR    : R → R → Prop
ltR    : R → R → Prop


NatAlt     : Type
NatAltle   : NatAlt → NatAlt → Prop
NatAltMax  : NatAlt → NatAlt → NatAlt
le_max_left  : forall x y, NatAltle x (NatAltMax x y)
le_max_right : forall x y, NatAltle y (NatAltMax x y)

add_comm  : forall x y, add x y = add y x
```

```
  add_assoc : forall x y z, add (add x y) z = add x (add y z)
  add_zero  : forall x, add x zero = x
  add_opp   : forall x, add x (opp x) = zero
  opp_add   : forall x y, opp (add x y) = add (opp x) (opp y)
  mul_comm  : forall x y, mul x y = mul y x
  mul_assoc : forall x y z, mul (mul x y) z = mul x (mul y z)
  mul_one   : forall x, mul x one = x

  le_refl   : forall x, leR x x
  le_trans  : forall x y z, leR x y → leR y z → leR x z
  add_le_add : forall a b c d, leR a b → leR c d → leR (add a c) \\
  (add b d)

  abs_nonneg : forall x, leR zero (abs x)
  abs_triangle : forall x y, leR (abs (add x y)) (add (abs x) (abs y))
  abs_opp : forall x, abs (opp x) = abs x
  abs_sub_symm : forall x y, abs (add x (opp y)) = abs (add y (opp x))


  sub_decomp : forall x y z, add x (opp z) = add (add x (opp y)) \\
  (add y (opp z))
  sub_eq_zero : forall x y, add x (opp y) = zero → x = y

  eq_of_forall_eps2 : forall x, (forall eps, ltR zero eps → leR (abs x) \\
  (add eps eps)) → x = zero

  namespace Limits

  variable {R : Type} [AR : AbsField R]

  def sub (x y : R) : R := AbsField.add x (AbsField.opp y)

  def limit (u : (AbsField.NatAlt (R := R)) → R) (l : R) : Prop :=
  forall eps : R, AbsField.ltR AbsField.zero eps →
  exists N : AbsField.NatAlt (R := R),
  forall n : AbsField.NatAlt (R := R),
  AbsField.NatAltle (R := R) N n →
  AbsField.leR (AbsField.abs (sub (u n) l)) eps

  theorem abs_sub_triangle (x y z : R) :
  AbsField.leR (AbsField.abs (sub x z)) (AbsField.add (AbsField.abs \\
  (sub x y)) (AbsField.abs (sub y z))) := by

  have : sub x z = AbsField.add (sub x y) (sub y z) := sub_decomp x y z
  simpa [this] using AbsField.abs_triangle (sub x y) (sub y z)

  theorem limit_unique (u : (AbsField.NatAlt (R := R)) → R) (l m : R) :
  limit u l → limit u m → l = m :=
  by
  intro Hl Hm

  have Hbound' : forall eps, AR.ltR AR.zero eps → AR.leR (AR.abs (sub l m)) \\
  (AR.add eps eps) := by
  intro eps Heps
```

```
  rcases Hl eps Heps with ⟨N1, HN1⟩
  rcases Hm eps Heps with ⟨N2, HN2⟩
  let N := AbsField.NatAltMax (R := R) N1 N2
  have H1 : AbsField.leR (AbsField.abs (sub (u N) l)) eps := by
  apply HN1
  exact AbsField.le_max_left (R := R) _ _
  have H2 : AbsField.leR (AbsField.abs (sub (u N) m)) eps := by
  apply HN2
  exact AbsField.le_max_right (R := R) _ _

  have Htri : AbsField.leR (AbsField.abs (sub l m)) (AbsField.add \\
  (AbsField.abs (sub l (u N))) (AbsField.abs (sub (u N) m))) := by
  simpa using abs_sub_triangle l (u N) m


  have H1' : AbsField.leR (AbsField.abs (sub l (u N))) eps := by

  simpa [sub, AbsField.abs_sub_symm (u N) l] using H1
  exact AbsField.le_trans _ _ _ Htri (AbsField.add_le_add _ _ _ _ H1' H2)

  have Hz : sub l m = AbsField.zero := AbsField.eq_of_forall_eps2 (sub l m) Hbound'

  exact AbsField.sub_eq_zero l m (by simpa [sub] using Hz)
  end Limits
```

A.5. Translation to SSReflect

We illustrate the Rocq → SSReflect translation on a lemma extracted from CoRN. Babel takes as input the proof term and produces a short SSReflect script that closes the goal under the same context.

Lemma (CoRN).

```
  Lemma CR_nonZero_as_Cauchy_IR_nonZero_1 :
  forall (x:CR), (CRapartT x 0)%CR ->
  Dom (f_rcpcl' _) (CRasCauchy_IR x).
```

Proof term.

```
  CR_nonZero_as_Cauchy_IR_nonZero_1 =
  fun (x : CR) (x_ : (x >< ' 0%Q)%CR) =>
  ap_wdr_unfolded Cauchy_IR (CRasCauchy_IR x) (CRasCauchy_IR (' 0%Q)%CR) [0]
  (CR_ap_as_Cauchy_IR_ap_1 x (' 0%Q)%CR x_)
  (CR_inject_Q_as_Cauchy_IR_inject_Q 0)
  : forall x : CR,
  (x >< ' 0%Q)%CR ->
  Dom (f_rcpcl' Cauchy_IR) (CRasCauchy_IR x)

  Arguments CR_nonZero_as_Cauchy_IR_nonZero_1 x _
```

Original Rocq script.

```
  intros x x_.
```

```
  change ((CRasCauchy_IR x)[#][0]).
  stepr (CRasCauchy_IR 0%CR).
  apply CR_ap_as_Cauchy_IR_ap_1.
  assumption.
  eapply CR_inject_Q_as_Cauchy_IR_inject_Q.
  Defined.
```

Babel (SSReflect) script from the term.

```
  move=> x_; rewrite /CRapartT; apply: CR_ap_as_Cauchy_IR_ap_1 => /=.
```

A.6. Translation from Lean to Rocq

We illustrate the Lean → Rocq translation on a lemma whose Lean proof uses the `simp` tactic. Babel takes as input the proof term and the goal (it does not see the Lean proof), and reconstructs the explicit calls to lemmas such as `map_smul` and `smul_zeroV` that `simp` applies implicitly.

Lemma (Lean).

```
  theorem ker_smul {L : LMap (R:=R) V W} {a : R} {x : V} :
  ker L x → ker L (VSpace.smul (R:=R) a x) :=
  by
  intro hx
  unfold ker at *
  simp [hx]
```

Proof term.

```
  ker_smul =
  fun (VS2 : VSpace) (L : Lin) (a : R0) (x : V0) (hx : ker L x) =>
  eq_ind_r (fun v : V0 => v = zeroV)
  (eq_ind_r (fun v : V0 => (a •V v) = zeroV)
  (eq_ind_r (fun v : V0 => v = zeroV) eq_refl (smul_zeroV a)) hx)
  (map_smul L a x)
  : forall {VS2 : VSpace} (L : Lin) (a : R0) (x : V0),
  ker L x -> ker L (a •V x)
```

Original Lean script.

```
  intro hx
  unfold ker at *
  simp [hx]
```

Babel (Rocq) reconstruction. The beginning of the first reasoning block (initial hypotheses and goal) is hardcoded by us to provide context; it is *not* generated by the model.

```
  Okay, let's try to transform this proof term into a sequence of Coq tactics.
  First, let's write down the hypotheses and the initial goal (after the "|-")
  given by the Coq proof assistant:
```

```
F  : Field
VS1, VS2  : VSpace
L  : Lin
a  : R0
x  : V0
|- ker L x -> ker L (a •V x)
```

The proof term starts with fun {R} ... {x} hx => .... This is a lambda abstraction that binds the variable hx. In a Coq proof, this corresponds to introducing an assumption. The type of this assumption is the premise of the implication in the goal, which is ker L x. So, the first logical step is to introduce this assumption into our proof context. The tactic for this is intros. The lambda term gives it the name hx, so I'll use that. This will change our goal to the conclusion of the implication, which is ker L (a •V x). </think>
\box{intros hx.}

<think> Now, my proof state is:
F, VS1, VS2, L, a, x : ...
hx : ker L x
|- ker L (a •V x)

The goal is ker L (a •V x). This is a definition. To make progress, I need to understand what I'm trying to prove. ker is the kernel of a linear map, which means it's the set of vectors that the map sends to zero. So, ker L (a •V x) should mean L (a •V x) = 0. To make the proof term easier to work with, I should unfold this definition. The tactic for unfolding definitions in the goal is unfold. Let's apply that to ker in the goal. </think>
\box{unfold ker in |- *.}

<think> The goal has now unfolded to L (a •V x) = Lin.VSpace.zeroV R. The lambda term contains L.map_smul a x. This looks like a lemma about linear maps and scalar multiplication. It probably states that L (a •V x) = a •V (L x). This seems like a very useful fact, as it lets me distribute the linear map L over the scalar multiplication •V. My current goal's left-hand side is L (a •V x), which is the perfect form to apply this lemma. The tactic rewrite is used to apply an equality to the goal. Since map_smul is a lemma of the form LHS = RHS, rewrite map_smul will replace LHS with RHS. I expect this to change my goal to a •V (L x) = Lin.VSpace.zeroV R. This looks like a promising simplification. </think>
\box{rewrite map_smul.}

<think> My goal is now a •V (L x) = Lin.VSpace.zeroV R. Let's look at my hypotheses. I have hx : ker L x. This is still using the ker definition. It would be helpful to have this hypothesis in its unfolded form as well, L x = Lin.VSpace.zeroV R. This will make it easier to use hx to rewrite parts of my goal. I'll use the unfold tactic again, this time on the hypothesis hx. </think>
\box{unfold ker in hx.}

```
<think> Now my context contains hx : L x = Lin.VSpace.zeroV R, and my goal is
a •V (L x) = Lin.VSpace.zeroV R. This is great. The term L x in my goal appears
exactly as the left-hand side of the equality in my hypothesis hx. This is a
perfect opportunity to use the rewrite tactic. By doing rewrite hx, I can
replace L x in the goal with Lin.VSpace.zeroV R. I expect this to transform
my goal into a •V (Lin.VSpace.zeroV R) = Lin.VSpace.zeroV R. </think>
\box{rewrite hx.}

<think> The goal is now a •V (Lin.VSpace.zeroV R) = Lin.VSpace.zeroV R. Looking
at the lambda term, I see a reference to Lin.VSpace.smul_zero a. This must be
the lemma for a •V 0 = 0. My goal's left-hand side, a •V (Lin.VSpace.zeroV R),
is precisely the form required by this lemma. Once again, rewrite is the tactic
to use. Applying rewrite smul_zeroV should simplify the left-hand side of my
goal to Lin.VSpace.zeroV R, making the
goal Lin.VSpace.zeroV R = Lin.VSpace.zeroV R. </think>
\box{rewrite smul_zeroV.}

<think> After the last rewrite, my goal is Lin.VSpace.zeroV R = Lin.VSpace.zeroV R.
This is an equality of a term with itself. This is a trivially true statement. There
are several tactics to solve such a goal. reflexivity is the most direct one, as it
is designed specifically for proving t = t. The lambda term structure Eq.trans ...
(eq_self ...) confirms that the proof concludes by showing both sides are the same
thing. Therefore, reflexivity is the final step to close the proof. </think>
\box{reflexivity.}
```